

Yet Another Way to Analyze Noise

An updated Python-Based Framework of Merchant et al. 2015 for Long-Term Underwater Noise Processing

K.J. Palmer^{*1}

¹NOAA Pacific Island Fisheries Science Center Affiliate with Ocean Associates, Inc.

March 19, 2026

Abstract

Ambient noise influences both the detectability of marine animals and the acoustic conditions they experience, making reliable long-term noise metrics essential for passive acoustic monitoring (PAM). Modern monitoring programs routinely produce multi-terabyte archives, creating a need for scalable and reproducible processing workflows.

This technical note presents Yet Another Way to Analyze Noise (YAWN), a lightweight Python framework for long-term underwater noise analysis based on the methodology of Merchant et al. (2015). YAWN streams audio files in fixed-duration blocks, computes calibrated power spectral densities, and summarizes spectra using broadband, third-octave, decade, and hybrid millidecade metrics. Results are stored in daily HDF5 files that preserve timestamps, frequency axes, and processing parameters required for reproducibility.

To validate the implementation, hybrid millidecade spectra produced by YAWN were compared with those generated by the PyPAM package using identical analysis settings. After harmonizing calibration conventions, the two implementations agree within approximately 1 dB across the analysis band, with a median offset of 0.45 dB. These results demonstrate that the YAWN implementation reproduces established hybrid millidecade metrics while providing a flexible Python-based workflow suitable for large acoustic archives and integration with modern analysis pipelines.

1 Introduction

The ambient noise in the ocean affects both the behavior of marine animals and the performance of passive acoustic monitoring (PAM) systems. Elevated noise levels can mask biologically important signals, reduce detection ranges for PAM deployments, and complicate efforts to detect and protect threatened populations. Regulatory frameworks, including national and international initiatives that treat underwater noise as an environmental indicator, further motivate consistent and transparent methods for estimating long-term noise metrics.

Over the last decade, long-term PAM programs have expanded in scope, duration, and spatial coverage. Multi-year deployments on fixed recorders, gliders, and other platforms routinely produce data volumes that are difficult to manage with legacy workflows. At the same time, the need for

^{*}kaitlin.palmer@noaa.gov

reproducible open methods has grown, both for scientific synthesis and for regulatory contexts where traceable methodologies are essential.

Hybrid millidecade bands are a useful compromise between spectral resolution and storage cost [1]. The approach preserves fine, approximately linear resolution at low frequencies and transitions to logarithmically spaced bands at higher frequencies, allowing long-term spectral statistics to be stored compactly without discarding biologically meaningful structure.

There are a plethora of free and paid services used to calculate noise metrics including PAMGuard [2], Triton, MANTA [3], and PAMGuide [4]. Each of these systems incorporate various advantages and limitations. Bioacousicians often find themselves in need of modification for their own specific data needs. For instance, Triton and its associate Remoras are also free and there is a compiled version that does not require MATLAB. However, any customization does require a MATLAB license, which is frequently cost prohibitive, especially for researchers in developing countries. PAMGuard is both free and an industry standard. However, JAVA is not commonly known among biologists, making it challenging to troubleshoot without involving a small, but dedicated team of maintainers. Pypam is another free, open-source, and written in python. It contains multiple useful features for measuring soundscapes. This polished repository has been professionally developed and has been used by SanctSound. However, the nested dependencies that result in rigid data structure make it challenging to integrate custom calibration systems. It also cannot presently handle duty-cycled data. Finally, PAMGuide was written in both R and MATLAB and includes a Matlab-GUI, allowing for user-friendly interface. The paper was well received, with over 200 citations, in no small part because of the well-documented and published code provided by the author. However, it too has limitations that have required extensive modifications for many long-term noise projects. Principle among the limitations the speed of analysis and the storage options initially provided (either mat files or .csv files). These become which become impractical for multi-year or multi-instrument datasets.

Here we present Yet Another Way to Analyze Noise, a package based on [4]. The present work builds on PAMGuide by 1) exporting metrics to HDF5 files 2) Implementing it in freely available Python software 3) saving hybrid millidecade bands rather than full PSD matrices and 4) Allowing raw data to be pulled from google storage. Similar to [4], the YAWN framework is intentionally lightweight and organized as a single module to facilitate modification and inspection.

The Python implementation of YAWN is openly available at SPACIOUS-NoiseProcessing. To demonstrate that the YAWN implementation produces results that are scientifically interchangeable with existing tools, provided that calibration conventions are aligned, hybrid milidecade bands are directly compared to outputs from the PyPam package.

A versioned archival release will be made available via Zenodo once the manuscript is deposited on arXiv.

2 Comparisons

To ensure that YAWN reproduces established hybrid millidecade metrics, its output was compared with the PyPAM reference implementation. A glider-mounted hydrophone recording was processed with both tools using matching FFT parameters, averaging intervals, and analysis bands. The comparison required careful alignment of calibration conventions. YAWN applies calibration through S_i , the hydrophone sensitivity in dB re 1 V/ μ Pa. With $S_i = 0$, voltage PSDs are converted using a neutral 1 V/ μ Pa mapping.

2.1 YAWN Architecture

YAWN processes long-term acoustic recordings by reading audio files sequentially, extracting time frequency representations, converting them to acoustic pressure units, and aggregating the results into daily HDF5 files. The application identifies input files either locally or in cloud storage and infers their timestamps directly from filenames using configurable regular expressions. When timestamps cannot be extracted, the file metadata provides a fallback.

Each file is streamed in blocks to avoid loading multi-gigabyte recordings into memory. Audio files are processed in sequential blocks of fixed duration (typically 30 s), allowing arbitrarily long recordings to be analyzed while maintaining a constant memory footprint. After optional DC removal, in which the mean value of each audio block is subtracted to remove sensor offsets and very low frequency bias, YAWN computes power spectral densities (PSDs) using the `scipy.signal.spectrogram` function with user-specified FFT size, overlap, and temporal averaging interval. PSDs are initially expressed in voltage units and are subsequently converted to acoustic pressure using a scalar or frequency-dependent sensitivity parameter, `Si`, which specifies the hydrophone response in dB re 1 V/ μ Pa. Setting `Si = 0` yields a neutral calibration of 1 V/ μ Pa, although a frequency-sensitivity CSV may also be provided when instrument responses vary with frequency.

Once calibrated, the PSDs are used to derive several acoustic metrics including broadband levels, third-octave bands, decade bands, and hybrid millidecade spectra. These quantities represent different integrations of the calibrated PSD and therefore do not share identical physical units.

Hybrid millidecade spectra retain the spectral-density formulation and are expressed in units of dB re 1 Pa²/Hz. In contrast, broadband levels, third-octave bands, and decade bands represent acoustic energy integrated across a finite frequency band and are therefore reported as sound pressure levels in units of dB re 1 Pa. This distinction reflects whether the metric preserves the spectral-density representation or integrates spectral power across a bandwidth.

Table 1: Units associated with the acoustic metrics produced by YAWN.

Metric	Units
Hybrid millidecade spectra	dB re 1 Pa ² /Hz
Broadband sound pressure level	dB re 1 Pa
Third-octave band levels	dB re 1 Pa
Decade band levels	dB re 1 Pa

These metrics, together with their associated frequency axes and timestamps, are written to a daily HDF5 file named for the project name and date in year, month, day format. Time bins are referenced to the start of the recording day, allowing multiple files from the same day to be aggregated onto a consistent temporal axis. This approach simplifies long-term analysis by ensuring that timestamps remain continuous even when recordings are split across many source files.

Data Storage Each HDF5 file is organized under a single deployment group (e.g. CalCurCEAS_2024). Time information is stored in `DateTime`, while all processing and spectral estimation settings are captured as attributes within the `Parameters` group. Spectral products are stored as separate datasets, including broadband levels, third-octave bands, decade bands, and hybrid millidecade spectra, with corresponding frequency axes stored explicitly.

Datetime Handling YAWN automatically infers recording timestamps from filenames using a small library of configurable regular expressions. Common logger conventions (e.g. AMAR, Sound-

Trap, and simple `yyyymmdd_HHMMSS` formats) are detected from the first file in the series and parsed into absolute datetimes; the same pattern is then applied consistently to the remaining files. When no known pattern is found, YAWN falls back to the file modification time to ensure that every averaged spectrum is associated with a valid timestamp. This flexible, pattern-based approach avoids hard-coded filename assumptions and reduces reliance on external log files, providing flexibility when working with heterogeneous archives.

YAWN implicitly supports duty cycling by processing sequential files and bins timestamps.

Name	Regex pattern	Example fragment	strptime format
yyyymmdd_HHMMSS_fff	\d{8}_\d{6}_\d{3}	20241001_123045_123	%Y%m%d_%H%M%S_%f
yyyymmdd_HHMMSS	\d{8}_\d{6}	20241001_123045	%Y%m%d_%H%M%S
AMAR	\d{8}T\d{6}	20241001T123045	%Y%m%dT%H%M%S
SoundTrap_1	\d{9}\.\d{12}	202401011.123456789012	%Y%m%d%H%M%S%f
SoundTrap_2	\d{4}\.\d{12}	2024.123456789012	%Y%m%d%H%M%S%f
yymmdd-HHMMSS.fff	\d{6}-\d{6}\.\d{3}	241001-123045.123	%y%m%d-%H%M%S.%f
yymmdd_HHMMSS	\d{6}_\d{6}	241001_123045	%y%m%d_%H%M%S

Table 2: Pre-configured filename patterns used by YAWN to infer recording datetimes. The appropriate regular expression is inferred from the first file in the archive and applied to subsequent files; when no pattern matches, file modification time is used as a fallback.

Broadband Broadband sound pressure levels are derived from the calibrated power spectral density (PSD) by integrating spectral power across the analysis frequency band. The spectrogram is computed using the `scipy.signal.spectrogram` routine with density scaling, which produces a single-sided PSD estimate $S_{pp}(f)$ with units of $\mu\text{Pa}^2/\text{Hz}$.

For each time interval, the band-limited mean-square pressure is obtained by numerical integration of the PSD,

$$p_{\text{rms}}^2(t) = \int_{f_1}^{f_2} S_{pp}(f, t) df \approx \sum_{k=f_1}^{f_2} S_{pp}(f_k, t) \Delta f,$$

where Δf is the FFT frequency resolution and f_1 and f_2 define the analysis band. The broadband sound pressure level is then computed as

$$L_p(t) = 20 \log_{10} \left(\frac{p_{\text{rms}}(t)}{p_{\text{ref}}} \right),$$

where the reference pressure is $p_{\text{ref}} = 1 \mu\text{Pa}$.

This formulation yields a band-limited RMS sound pressure level derived directly from the calibrated spectral density estimate and is independent of the specific window function used during spectral estimation.

For comparison with legacy analyses produced by PAMGuide, YAWN optionally provides a compatibility mode that applies the window noise-power bandwidth normalization used in that software. This mode reproduces PAMGuide broadband levels but is not required for the physically consistent PSD-integration formulation used in the default implementation.

Third-Octave Level Computation Third-octave band levels in YAWN are derived from the calibrated power spectral density (PSD). After the spectrogram is computed, each time-averaged PSD $S_{pp}(f)$ is expressed in units of $\mu\text{Pa}^2/\text{Hz}$ on the FFT frequency grid. Third-octave bands are defined using logarithmically spaced center frequencies that follow a base-10 progression, with approximately ten bands per decade of frequency. This approach is not ANSI standard as it is calculated with hard-cutoffs in the frequency domain. However, it drastically reduces processing time. As such and as with all other metrics, users should be judicious about comparing noise levels across different systems.

For each band k with center frequency f_c , the lower and upper band edges are defined as

$$f_{k,1} = f_c \times 10^{-0.05}, \quad f_{k,2} = f_c \times 10^{0.05}.$$

All FFT bins whose center frequencies fall within the interval $[f_{k,1}, f_{k,2}]$ are identified. The band power is then computed by summing the calibrated PSD values across the corresponding frequency bins,

$$P_k(t) = \sum_{f_i \in [f_{k,1}, f_{k,2}]} S_{pp}(f_i, t).$$

The resulting band-integrated power is converted to sound pressure level as

$$L_k(t) = 10 \log_{10} \left(\frac{P_k(t)}{p_{\text{ref}}^2} \right),$$

where $p_{\text{ref}} = 1 \mu\text{Pa}$. This method yields third-octave band sound pressure levels derived from frequency-domain integration of the calibrated PSD. Because the band limits are applied as hard frequency boundaries rather than ANSI filterbank responses, the resulting levels represent spectral-band integrations rather than strict ANSI S1.11 filterbank measurements.

This approach is computationally efficient because it operates on the already computed PSD matrix and avoids additional time-domain filtering operations.

Decade Bands Decade-band levels are derived from the calibrated power spectral density (PSD) by integrating spectral power across logarithmically spaced frequency intervals that span one tenth of a decade in frequency. The bands follow a base-10 progression such that center frequencies are spaced by a constant multiplicative factor corresponding to ten bands per decade.

For each band k with center frequency f_k , the lower and upper band edges are defined in log-frequency space as

$$f_{k,1} = f_k \times 10^{-0.05}, \quad f_{k,2} = f_k \times 10^{0.05}.$$

All FFT bins whose center frequencies fall within the interval $[f_{k,1}, f_{k,2}]$ are identified. Band power is computed by summing the calibrated PSD values across these bins,

$$P_k(t) = \sum_{f_i \in [f_{k,1}, f_{k,2}]} S_{pp}(f_i, t) \Delta f,$$

where Δf is the FFT bin width. The decade-band sound pressure level is then obtained as

$$L_k(t) = 10 \log_{10} \left(\frac{P_k(t)}{p_{\text{ref}}^2} \right),$$

with reference pressure $p_{\text{ref}} = 1 \mu\text{Pa}$. This procedure produces band-integrated sound pressure levels that summarize spectral energy within each logarithmic frequency interval.

Hybrid Millidecade Bands Hybrid millidecade bands provide a compact representation of long-term soundscapes by combining linear frequency resolution at low frequencies with logarithmically spaced bands at higher frequencies [1]. The calibrated PSD matrix $S_{pp}(f, t)$ is first defined on the FFT frequency grid.

From the lowest analysis frequency up to a crossover frequency f_{cross} (currently 435 Hz), each FFT bin is treated as an independent band, preserving the native linear frequency resolution of the spectrogram. Above this crossover, the algorithm constructs logarithmically spaced bands using millidecade spacing, defined by a base-10 exponential progression.

If N bands are defined per decade, the center frequency of band k is

$$f_k = f_0 \times 10^{k/N},$$

with band edges determined by half a band in log-frequency space. For each band with limits $[f_{\text{low}}, f_{\text{high}}]$, the algorithm identifies all FFT bins whose center frequencies fall within the band and sums their power in linear units,

$$P_{\text{band}}(t) = \sum_{f \in [f_{\text{low}}, f_{\text{high}}]} S_{pp}(f, t) \Delta f.$$

The summed power is then normalized by the effective band width to obtain an average spectral density,

$$S_{\text{band}}(t) = \frac{P_{\text{band}}(t)}{\max(f_{\text{high}} - f_{\text{low}}, \Delta f)},$$

and converted to decibels as

$$L_{\text{band}}(t) = 10 \log_{10} \left(\frac{S_{\text{band}}(t)}{p_{\text{ref}}^2} \right),$$

where $p_{\text{ref}} = 1 \mu\text{Pa}$. If a band contains no FFT bins, the PSD value at the nearest FFT frequency is used; if exactly one bin is present, that bin defines the band directly. The resulting hybrid millidecade matrix contains one spectrum per averaging interval and one value per band and is written to the HDF5 output together with the band frequency definitions.

Visualization and Long-Term Spectrograms In addition to computing hybrid millidecade and other band-integrated noise metrics, YAWN provides plotting utilities through the object. These functions operate directly on the daily HDF5 outputs, avoiding the need to reread raw audio. Long-term spectrograms (long-term spectral averages; LTSAs) can be generated with user-defined temporal averaging windows and frequency axes.

The `plot_ltsa` routine constructs LTSAs by aggregating calibrated power spectral densities over arbitrary averaging intervals (e.g. 15 minutes, 1 hour, 4 hours), returning spectral density in dB re $1 \mu\text{Pa}^2/\text{Hz}$ on a user-specified frequency grid. The frequency axis can be displayed either in linear frequency or in $\log_{10}$ frequency space, allowing low-frequency structure and higher-frequency bands to be visualized on the same panel. Complementary functions, such as `plot_milidecade_statistics`, summarize hybrid millidecade bands through percentile spectra, RMS spectra, or spectral probability density functions, again computed directly from the HDF5 archives. Together, these tools support rapid visual inspection of long-term soundscapes and diagnostic checks on processing settings without duplicating the underlying computations.

Pandas time-offset strings (e.g., 15min, 1h, 4h, 1D). These strings allow averaging intervals to be set with arbitrary temporal resolution without modifying the underlying processing code. Figure 1 shows an example LTSA for a single day, constructed from calibrated spectra averaged over 15-minute intervals and plotted on a $\log_{10}$ frequency axis. Figure 2 extends the same approach to a 30-day period, using 4-hour averaging windows to highlight broader temporal patterns while retaining biologically relevant spectral detail. Both examples are generated directly from the HDF5 archives using the `plot_ltsa` function, illustrating how users can tailor averaging periods and frequency scaling to the temporal and spectral resolution required by a given application.

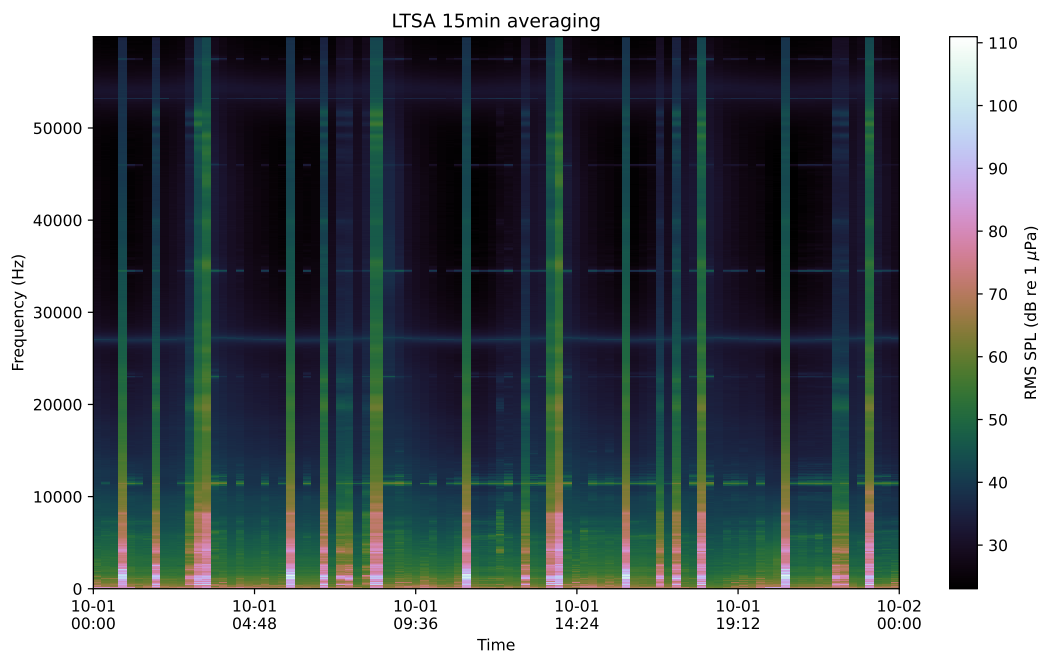


Figure 1: Example long-term spectral average (LTSA) for a single day, generated from calibrated spectra averaged over 15-minute intervals using the `plot_ltsa` function. Spectral density is shown in dB re 1 $\mu\text{Pa}^2/\text{Hz}$ on a $\log_{10}$ frequency axis, illustrating sub-daily variability in both level and spectral shape.

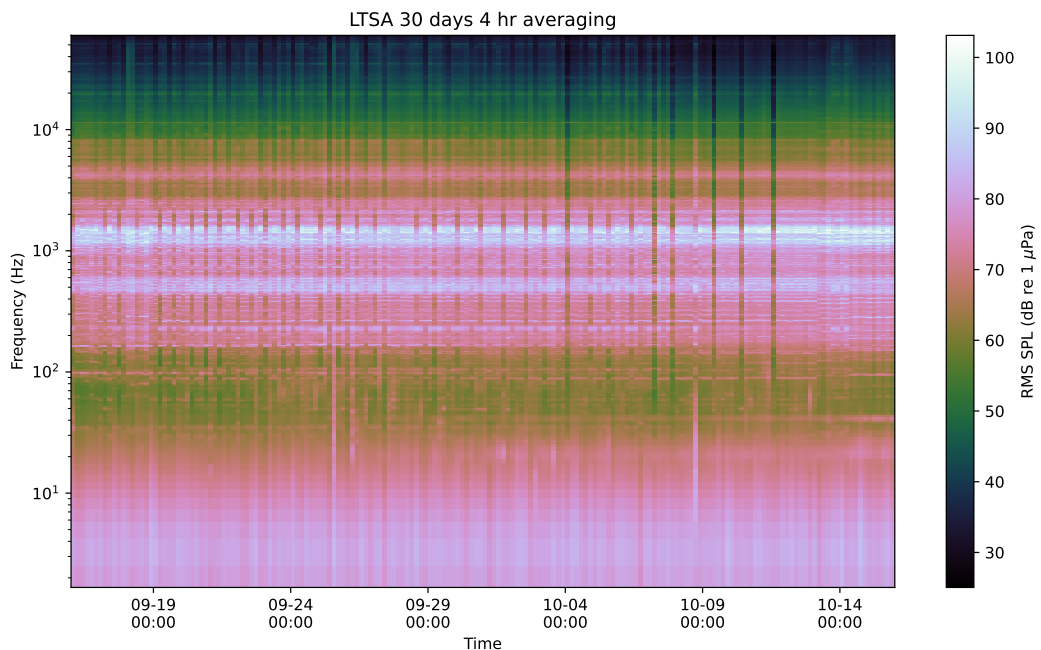


Figure 2: Example long-term spectral average (LTSA) for a 30-day period, with spectra averaged over 4-hour intervals. This panel highlights longer-term changes in ambient noise across the deployment while retaining sufficient spectral resolution for biologically relevant interpretation. As in Figure 1, the plot is generated directly from the HDF5 archives using the `plot_ltsa` function and displayed on a $\log_{10}$ frequency axis.

```
fig = plot_lttsa(groups,
                 averaging_period='4h',
                 titleText="LTSA 30 days 4 hr averaging",
                 freq_scaled=True,
                 log_freq=True)
```

2.2 PyPAM

PyPAM uses a different parameterization. Its `Hydrophone` object defines sensitivity and the peak-to-peak voltage corresponding to full-scale digital samples. When audio is read as floating-point values in $[-1, 1]$, the peak-to-peak range is two units; therefore, `Vpp = 2.0` must be specified. Using `Vpp = 1.0` would underestimate voltage by a factor of two and reduce acoustic power by 6 dB. For the comparison, PyPAM was configured with `sensitivity = 0` and `Vpp = 2.0`, yielding the same effective calibration as `Si = 0` in YAWN.

After processing, the hybrid millidecade spectra produced by each tool were averaged in linear space and converted back to decibels for comparison.

Data Storage PYPAM returns data as Xarrays that can easily be converted to NETCDF

Datetime Handling PYPAM is capable of handling continuous datetimes and pulls these data from phyhydrophone. Custom handling of date-times in audio files is not native.

Hybrid Millidecade Computation Hybrid millidecade bands combine linear-frequency resolution at low frequencies with logarithmically spaced bands at higher frequencies. The PSD matrix is first defined on the FFT frequency grid. From the lower analysis limit up to a crossover frequency of 435 Hz, each FFT bin is treated as its own narrow band, preserving the original linear resolution. Above the crossover, the algorithm constructs logarithmically spaced bands using fixed millidecade spacing, defined by a base-10 exponential progression. Each band therefore has a well-defined center frequency and frequency extent in log space.

For a given band, all FFT bins with centers falling within the band edges are identified. Their powers are summed in linear units, scaled by the FFT bin width, and normalized by the effective bandwidth of the hybrid band. The resulting quantity is an average spectral density across the band, which is then converted back to decibels. If a band contains no FFT bins, the algorithm evaluates the PSD at the nearest FFT frequency; if only one bin is present, its PSD defines the band directly. The completed hybrid millidecade matrix contains one spectrum per averaging interval and one value per hybrid band, and is stored in the HDF5 output along with the corresponding band definitions.

Hybrid millidecade bands in YAWN are computed from the calibrated PSDs in dB re $1 \mu\text{Pa}^2/\text{Hz}$, defined on the FFT frequency grid of the spectrogram.

For a given time-averaged PSD matrix $A(f, t)$ in dB, YAWN first defines a linear-frequency region from the lowest analysis frequency up to a crossover frequency f_{cross} (currently 435 Hz). In this region, each FFT bin is treated as its own band, with nominal centre frequency equal to the FFT bin center and band edges defined by $\pm\Delta f/2$, where Δf is the FFT bin width.

Above f_{cross} , the code constructs logarithmically spaced bands in “millidecades” of frequency. If the base is 10 and the number of bands per decade is N , the k -th band is defined by a centre frequency

$$f_k = f_0 \times 10^{k/N},$$

with lower and upper edges obtained by multiplying f_k by constant factors corresponding to half a band in log-space. The bands are truncated at the Nyquist frequency.

For each band with edges $[f_{\text{low}}, f_{\text{high}}]$, the algorithm identifies all FFT bins whose center frequencies fall within the band. Power is summed in linear units,

$$P_{\text{band}}(t) = \sum_{f \in [f_{\text{low}}, f_{\text{high}}]} 10^{A(f,t)/10} \Delta f,$$

and then normalized by the band width to obtain an average spectral density,

$$S_{\text{band}}(t) = \frac{P_{\text{band}}(t)}{\max(f_{\text{high}} - f_{\text{low}}, \Delta f)}.$$

Finally, the band-averaged spectral density is converted back to dB:

$$L_{\text{band}}(t) = 10 \log_{10} \left(\frac{S_{\text{band}}(t)}{p_{\text{ref}}^2} \right),$$

where $p_{\text{ref}} = 1 \text{ } \mu\text{Pa}$.

If a band contains no FFT bins, the algorithm falls back to the nearest FFT bin at the band centre. If a band contains exactly one bin, that bin's PSD is used directly. The resulting hybrid millidecade matrix has shape (time, band) and is stored in the HDF5 file alongside a matrix of band frequencies and edges.

Third Octave Level Computation As a result, differences of several decibels between YAWN/PAMGuide and PyPAM third-octave levels are expected and reflect fundamental methodological differences rather than implementation errors. For this reason, third-octave levels derived using YAWN are validated against PAMGuide outputs, while PyPAM third-octave results are treated as internally consistent spectral metrics.

Third-octave level calculation approaches YAWN, PYPAM, and PAMGuide compute Third-octave band sound levels using conceptually distinct approaches that are not mathematically equivalent, even when identical band center frequencies and averaging periods are used. In YAWN, third-octave levels are calculated using an ANSI-compliant time-domain filterbank approach. This is consistent with documentation in the 2022 update of PAMGuide (TOLf) but that does not appear to be functional. In this approach, the audio signal is passed through a bank of bandpass filters corresponding to standardized one-third-octave bands (ANSI S1.1-1986). For each band, the filtered signal is squared, averaged over time, and converted to a root-mean-square (RMS) pressure level before being expressed in decibels. This approach yields band-limited RMS sound pressure levels and explicitly accounts for filter roll-off and spectral leakage between adjacent bands, as defined by the ANSI standard. Let $x(t)$ denote the acoustic pressure time series. For each third-octave band k , the signal is passed through a bandpass filter with impulse response $h_k(t)$, producing

$$x_k(t) = (x * h_k)(t). \quad (1)$$

The band-limited sound pressure level is then computed as a root-mean-square (RMS) value over a time window T ,

$$L_k^{\text{ANSI}} = 10 \log_{10} \left(\frac{1}{T} \int_0^T x_k^2(t) dt \right). \quad (2)$$

In contrast, PyPAM derives third-octave levels using a frequency-domain integration approach. The power spectral density (PSD), $S_{xx}(f)$, is estimated using Fourier-based methods (e.g., Welch averaging). For each third-octave band k with lower and upper frequency limits $f_{k,1}$ and $f_{k,2}$, the band level is computed as

$$L_k^{\text{PSD}} = 10 \log_{10} \left(\int_{f_{k,1}}^{f_{k,2}} S_{xx}(f) df \right). \quad (3)$$

This fast approach uses on the pre-computed PSD and method applies hard frequency bounds in the spectral domain and integrates total spectral energy within each band,. The resulting values represent the integrated spectral power within each band rather than time-domain RMS pressure. Because this method applies hard frequency cutoffs in the spectral domain and does not implement ANSI filterbank responses, the resulting third-octave levels are not directly comparable to ANSI filterbank-based measurements. However, this approach is considerably faster than time domain processing necessary for the ANSI filterbanks.

Calibration and comparison Figure 3 and Figure 4 show the time-averaged hybrid millidecade spectra produced by YAWN and PyPAM for the glider and white noise recording, respectively. Binsize was set to 60 s, nfft was 180260 samples and overlap was 50%.

The spectral shapes matched closely across the full analysis band for both audio files. To quantify the agreement, the difference between the two spectra (YAWN minus PyPAM) was computed for each millidecade band. For the glider audio file, median offset was 0.45 dB. The 5th–95th percentile interval of the offset distribution ranged from 0.06 dB to 1.04 dB. The residual difference was spectrally flat and small in magnitude, consistent with expected implementation details such as window normalization and effective noise bandwidth.

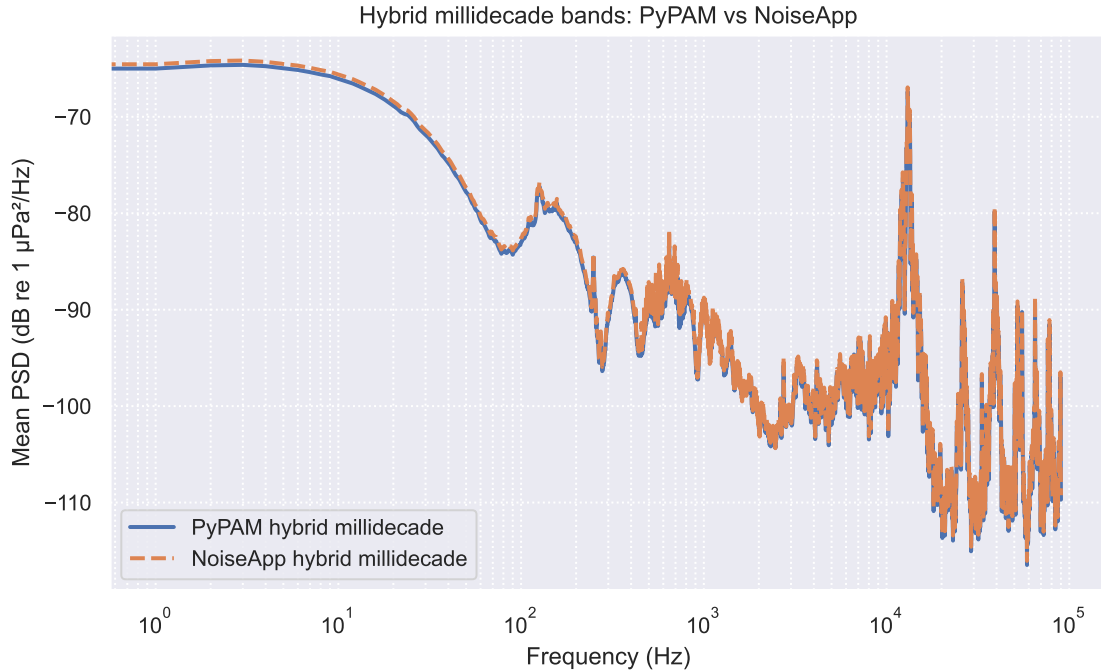


Figure 3: Time-averaged hybrid millidecade spectra for a glider-mounted hydrophone recording, computed with PyPAM and the Python-based YAWN implementation.

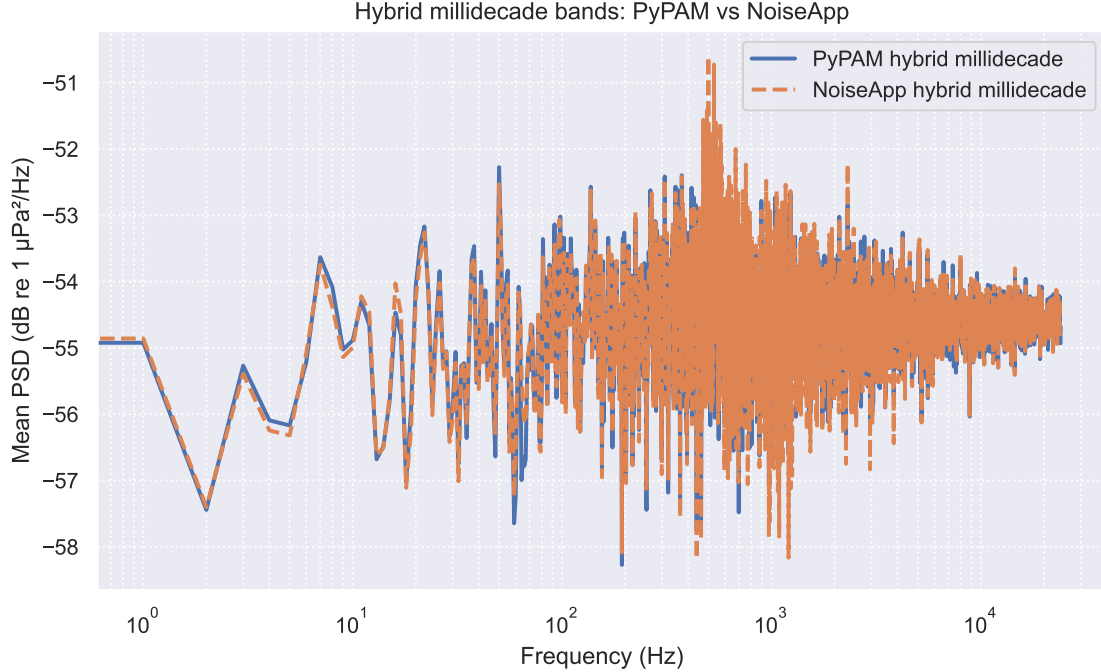


Figure 4: Time-averaged hybrid millidecade spectra for the synthetic white-noise recording, computed with PyPAM and the Python-based YAWN implementation.

2.3 PAMGuide

To validate broadband noise estimates produced by the YAWN NoiseApp framework, results were compared against those generated by PAMGuide [4], a widely used MATLAB-based tool for long-term ambient noise analysis. Both tools operate on overlapping, windowed segments of calibrated audio data and compute frequency-domain representations prior to band integration; however, differences in spectral normalization lead to systematic offsets in reported broadband levels. In addition, PAMGuide defines analysis timestamps at the start of each analysis window, whereas SciPy-based spectrogram implementations report timestamps at window centers; aligning these conventions was necessary to ensure that identical time segments were averaged in each tool.

Data Storage PAMGuide stores data as CSV files and must be re-run for each metric

Hybrid Millidecade Computation PAMGuide does not provide a hybrid millidecade option and this was therefor not evaluated

Broadband computation NoiseApp computes broadband noise levels by integrating a calibrated power spectral density (PSD) over a specified frequency band. Audio data are segmented using overlapping Hann windows, and the PSD is estimated using Welch’s method as implemented in `scipy.signal.spectrogram` with `scaling='density'`. This yields a single-sided PSD $S_{pp}(f)$ with units of $\mu\text{Pa}^2/\text{Hz}$, where window normalization is handled internally by the PSD estimator.

For each time bin, broadband mean-square pressure is computed as

$$p_{\text{rms}}^2 = \int_{f_1}^{f_2} S_{pp}(f) \, df \approx \sum_{k=f_1}^{f_2} S_{pp}(f_k) \Delta f, \quad (4)$$

where Δf is the frequency bin width and f_1 and f_2 define the analysis band. The broadband sound pressure level is then

$$L_p = 20 \log_{10} \left(\frac{p_{\text{rms}}}{p_{\text{ref}}} \right), \quad (5)$$

with reference pressure $p_{\text{ref}} = 1 \mu\text{Pa}$ for underwater recordings. This approach yields a physically interpretable band-limited RMS pressure that is independent of window choice, provided the PSD is correctly normalized.

PAMGuide follows the discrete Fourier transform (DFT)-based formulation described in Appendix S1 of Merchant et al. [4]. Audio data are divided into overlapping segments, multiplied by a window function $w[n]$ with scaling factor α , and transformed via the DFT. The single-sided power spectrum $P_{\text{ss}}(f)$ is formed as

$$P_{\text{ss}}(f_k) = 2 \left| \frac{X(f_k)}{N} \right|^2, \quad (6)$$

where $X(f_k)$ is the DFT magnitude and N is the segment length. PAMGuide computes the window noise power bandwidth

$$B = \frac{1}{N} \sum_{n=1}^N \left(\frac{w[n]}{\alpha} \right)^2, \quad (7)$$

which is used to normalize spectral density estimates.

For broadband noise, however, PAMGuide integrates the single-sided power spectrum directly without applying the $1/B$ correction:

$$L_p^{\text{PG}} = 10 \log_{10} \left(\frac{\sum_k P_{\text{ss}}(f_k)}{p_{\text{ref}}^2} \right) - S, \quad (8)$$

where S is the hydrophone sensitivity in dB re 1 V/ μPa . As a result, PAMGuide broadband levels are tied to the specific DFT and window normalization convention used, and may exhibit a window-dependent offset relative to PSD-integrated RMS estimates.

Third Octave Level Computation Initial publication of PAMGuide calculated 1/3rd octave bands by summing over the relevant PSD bands. In 2021 an additional method for calculating third octave bands, 'TOLF', was implemented. This changed which addressed a previously found bug as well as provided an ANSI compliant method of calculating third octave bands. This approach uses filter banks to calculate levels from the time domain. However, the presnet author has been unsuccessful in getting this method to run.

Calibration and comparison with PAMGuide To validate the Python implementation, hybrid millidecade spectra from YAWN were compared with those from the PyPAM package, using the two audio inputs with nominal analysis settings. The first audio file was from an underwater glider and sampled audio at 180,260 Hz and was 5 minutes long. The second audio file was the synthetic white noise file provided with the PAMGuide files. [4]. This file was 10 seconds long and sampled at 48 kHz.

The two systems differ in how they handle the relationship between digital sample values, volts, and acoustic pressure, so a fair comparison requires aligning their amplitude calibration conventions. In YAWN, the scalar `Si` (dB re 1 V/ μPa) specifies the conversion from voltage PSD to acoustic PSD. Setting `Si = 0` enforces a neutral calibration of 1 V/ μPa . For consistency, `Si = 0` was used in YAWN and PyPAM was configured with `sensitivity = 0` and `Vpp = 2.0`. Under these matched

settings, the spectra produced by the two tools are nearly identical at the plotted scale, with a median offset of 0.45 dB and a 5th–95th percentile range of 0.06 dB to 1.04 dB.

PyPAM expresses calibration through a `Hydrophone` object. Floating-point audio is assumed to lie in the range $[-1, 1]$, which spans two units peak-to-peak and therefore corresponds to $V_{pp} = 2.0$. Assigning `sensitivity = 0` dB re 1 V/ μ Pa reproduces the neutral calibration used in YAWN. If V_{pp} were instead set to 1.0, the voltage represented by the samples would be underestimated by a factor of two, shifting all PSD values downward by 6 dB.

With the calibration parameters harmonized, PYPAM’s hybrid millidecade bands method was run using the same FFT length, frequency bands, and time-bin sizes as YAWN. Time-averaged spectra for both implementations were then computed by averaging in the linear domain across time and converting the results to decibels.

3 Discussion

This study evaluated the YAWN noise processing framework by comparing its outputs to those from PyPAM and PAMGuide under matched analysis settings. Hybrid millidecade spectra generated by YAWN and PyPAM agreed to within approximately 1 dB across the analysis band, with a median offset of 0.45 dB. Broadband levels showed similar temporal structure across tools, with small systematic offsets attributable to differences in spectral normalization.

These results indicate that YAWN reproduces established noise metrics when calibration and processing parameters are aligned. Differences in digital scaling and voltage-to-pressure conversion introduce systematic offsets when not explicitly aligned, even when identical FFT parameters and band definitions are used and should be a reminder for users to consider these aspects carefully.

YAWN provides several practical advantages for large-scale passive acoustic monitoring. First, because YAWN can handle multiple time formats, it is able to handle duty-cycled recordings without issue. This feature is not yet available in the PyPAM implementation. Along the same lines, it is fairly straightforward to implement a novel timestamp format within YAWN that does not require creating custom hydrophones. The use of HDF5 storage enables efficient handling of multi-year datasets while preserving metadata required for reproducibility. Furthermore, these file structures are related to the NETCDF files created by PyPAM which should facilitate multi-tool comparison in the future. As with PyPAM, YAWN Sequential processing allows arbitrarily long recordings to be analyzed with a fixed memory footprint.

The plotting capacity of YAWN is limited but functional. Plotting of Spectral Probability Density and LTSAs is available in both PyPAM and YAWN. However, YAWN allows for users to define custom intervals over which to plot these intervals (e.g. days, weeks, months). This feature is not yet available in PyPAM.

However, the YAWN framework contains several limitations in comparison to both PAMGuide and PyPAM. First and foremost, PyPAM is capable of reporting multiple acoustic indices, time domain, and frequency domain metrics not included in the YAWN implementation. Some of these were explored in the comparison protocol but, at time of writing, many features either did not work or could not easily incorporate calibration. Second, frequency domain metrics including third-octave and decade bands are calculated from the spectrum and therefore do not follow ANSI standards as implemented in PyPAM. Preliminary testing also indicates that PyPAM is likely faster at processing large datasets though this has not been fully evaluated. Last PAMGuide includes native tools for calibrating hydrophones with piston mics that are not available in either YAWN or PyPAM.

Overall, YAWN provides a reproducible and scalable framework for long-term underwater noise

analysis that is consistent with established methods while addressing practical limitations in data handling and workflow integration. It provides yet another option for those looking to process large datasets with standard metrics Kait worked on a draft of the noise monitoring tool for assessing ambient noise from glider data available [here](#). This tool complements existing PyPAM architecture being developed by PAM-SI programs and addresses some of our immediate needs including being able to handle custom calibration values across all metrics (e.g. third-octave, decidecade, hybrid-millidecade, and broadband levels). It also handles duty-cycled data that are not yet able to be processed within the PyPAM system. The document compares and contrasts the systems and describes the cross validation efforts used to ensure consistency. Code, and example files are available on [github](#).

Table 3: Comparison of long-term underwater noise analysis frameworks (updated)

Feature	YAWN (NoiseApp)	PyPAM	PAMGuide
General use			
Primary language	Python	Python	MATLAB/R
Intended use	Long-term underwa- ter noise processing	Standards-driven acoustic metrics and soundscape analysis	Hydrophone calibra- tion; long-term un- derwater noise pro- cessing
Associated software or packages	SciPy, NumPy	SciPy, NumPy, Py- hydrophone	–
Storage	HDF5	xarray (recom- mended NetCDF output)	CSV
Cloud computing	Processes data stored on Google Cloud	Native support with Amazon S3 buckets	None
Metric calculation			
Calibration	Linear and frequency-response	Linear and frequency-response	Linear only
Hybrid millidecade	Derived from PSD via band aggregation	Derived from PSD via band aggregation	N/A
Third-octave levels	Frequency-domain PSD integration (not ANSI-compliant)	Frequency-domain integration (ANSI- aligned band defini- tions)	ANSI filterbank op- tion (TOLf); earlier versions used spec- tral integration
Decade bands	PSD integration over frequency bins	–	–
Broadband	PSD integration over frequency bins	PSD integration over frequency bins	PSD integration over frequency bins
Additional reported metrics (not tested)	None	Spectrogram, PSD, power spectrum, spectral probability density (SPD)	–
Native plotting	LTSA, spectral prob- ability density	–	–
Status and limitations			
Maintenance status	Active development	Actively maintained	Largely legacy
Known issues	Not ANSI-compliant (frequency-domain band integration)	Challenges with duty-cycled data; requires struc- tured hydrophone metadata; limited flexibility for custom post hoc calibration workflows	Limited accessibility and stability of newer third-octave implementations

4 Conclusion

This technical note introduces YAWN, a Python workflow for long-term underwater noise processing based on Merchant et al. (2015). YAWN streams audio in blocks, computes Welch PSDs, converts spectra to acoustic units, and summarizes them as broadband, third-octave, decade, and hybrid millidecade metrics. Outputs are stored as daily HDF5 files that include timestamps, frequency axes, and the processing settings needed to reproduce the analysis. The implementation is validated by comparing hybrid millidecade spectra to those produced by the established PyPAM package under matched analysis settings. With consistent calibration, the two tools agree within approximately 1 dB across the analysis band, with a median offset of 0.45 dB. The note also demonstrates plotting utilities that operate directly on the HDF5 archives, including long-term spectrograms with user-defined averaging windows.

References

- [1] Samuel B. Martin, Brian J. Gaudet, Holger Klinck, Patrick J. Dugan, Jennifer L. Miksis-Olds, David K. Mellinger, David A. Mann, Olaf Boebel, Curtis C. Wilson, Douglas W. Ponirakis, et al. Hybrid millidecade spectra: A practical format for exchange of long-term ambient sound data. *JASA Express Letters*, 1(1):011202, 2021. doi: 10.1121/10.0003249.
- [2] Douglas Gillespie, DK Mellinger, JONATHAN Gordon, David McLaren, PAUL Redmond, Ronald McHugh, PW Trinder, XY Deng, and A Thode. Pamguard: Semiautomated, open source software for real-time acoustic detection and localisation of cetaceans. *Journal of the Acoustical Society of America*, 30(5):54–62, 2008.
- [3] Jennifer L Miksis-Olds, Peter J Dugan, S Bruce Martin, Holger Klinck, David K Mellinger, David A Mann, Dimitri W Ponirakis, and Olaf Boebel. Ocean sound analysis software for making ambient noise trends accessible (manta). *Frontiers in Marine Science*, 8:703650, 2021.
- [4] Nathan D Merchant, Kurt M Fristrup, Mark P Johnson, Peter L Tyack, Matthew J Witt, Philippe Blondel, and Susan E Parks. Measuring acoustic habitats. *Methods in Ecology and Evolution*, 6(3):257–265, 2015.